

Orange

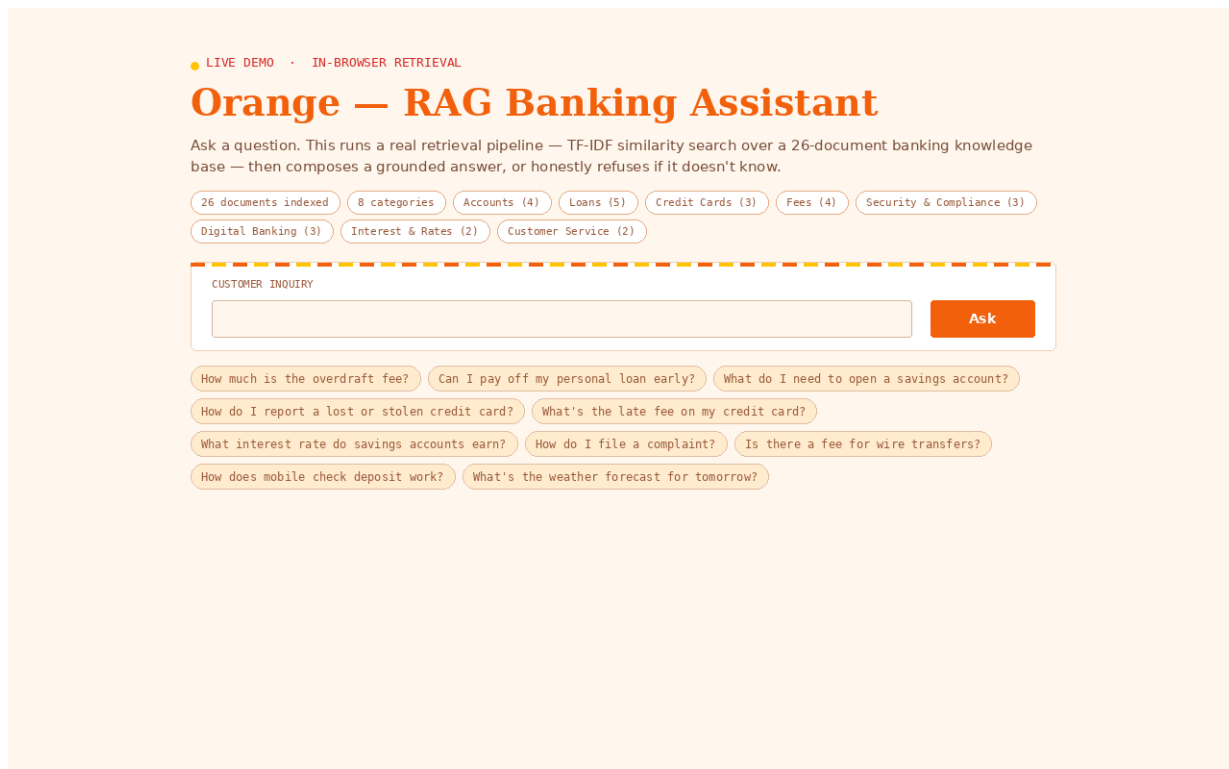
RAG-Based Banking Assistant — Demo Walkthrough Guide

This guide walks through exactly what to do to run the Orange demo ([orange_demo.html](#)) and explains what's happening behind the scenes at each step: a real, in-browser Retrieval-Augmented Generation (RAG) pipeline searching a 26-document banking knowledge base.

1 Open the app

Open [orange_demo.html](#) in any modern browser (double-click the file, or drag it into a browser window). No installation, server, or internet connection is required — everything, including the knowledge base and the retrieval logic, runs client-side in a single HTML file.

At the top you'll see the header, a short description of how the pipeline works, and a row of stat pills confirming what's loaded: 26 documents across 8 categories (Accounts, Loans, Credit Cards, Fees, Security & Compliance, Digital Banking, Interest & Rates, Customer Service). Below that is the query box and a set of example question chips you can click instead of typing.



Initial state: header, live stats, query box, and example chips.

2 Ask a question

Type a banking question into the **Customer inquiry** box, or click one of the example chips to fill it in automatically. Then click the orange **Ask** button (or press Enter).

Try a few different ones — the chips cover every category in the knowledge base, for example:

- “How much is the overdraft fee?”
- “What do I need to open a savings account?”
- “How do I report a lost or stolen credit card?”

● LIVE DEMO · IN-BROWSER RETRIEVAL

Orange — RAG Banking Assistant

Ask a question. This runs a real retrieval pipeline — TF-IDF similarity search over a 26-document banking knowledge base — then composes a grounded answer, or honestly refuses if it doesn't know.

CUSTOMER INQUIRY

How much is the overdraft fee? Ask

How much is the overdraft fee? Can I pay off my personal loan early? What do I need to open a savings account?

How do I report a lost or stolen credit card? What's the late fee on my credit card?

What interest rate do savings accounts earn? How do I file a complaint? Is there a fee for wire transfers?

How does mobile check deposit work? What's the weather forecast for tomorrow?

A question typed into the inquiry box, ready to submit.

3 Watch the retrieval pipeline run

After you click Ask, the page runs a real retrieval pipeline directly in your browser (no server calls, no external API):

Step 01 — Tokenize: your question is lower-cased and split into meaningful terms (common words like “the” and “is” are filtered out).

Step 02 — Retrieve: a TF-IDF vector is built for your question and compared, using cosine similarity, against every document in the knowledge base. The top 3 matches are shown as ranked “line items” with a relevance score and a score bar.

● LIVE DEMO · IN-BROWSER RETRIEVAL

Orange — RAG Banking Assistant

Ask a question. This runs a real retrieval pipeline — TF-IDF similarity search over a 26-document banking knowledge base — then composes a grounded answer, or honestly refuses if it doesn't know.

CUSTOMER INQUIRY

How much is the overdraft fee? Ask

01 QUERY RECEIVED & TOKENIZED
5 content terms extracted -> vectorized (TF-IDF)

02 TOP-3 CHUNKS RETRIEVED (COSINE SIMILARITY)

FEES Overdraft fees	42.0%
FEES ATM fees	11.5%
FEES Foreign transaction fees	7.8%

Steps 01–02: tokenization, then the top-3 retrieved sources ranked by relevance.

4 Read the grounded answer

Step 03 — Answer: if the top match scores above the confidence threshold, the app composes an answer using only the retrieved document's content, and shows exactly which source it came from (“Grounded in: ...”) so you can verify it yourself.

LIVE DEMO · IN-BROWSER RETRIEVAL

Orange — RAG Banking Assistant

Ask a question. This runs a real retrieval pipeline — TF-IDF similarity search over a 26-document banking knowledge base — then composes a grounded answer, or honestly refuses if it doesn't know.

CUSTOMER INQUIRY

How much is the overdraft fee?

Ask

01 QUERY RECEIVED & TOKENIZED

5 content terms extracted -> vectorized (TF-IDF)

02 TOP -3 CHUNKS RETRIEVED (COSINE SIMILARITY)

FEES Overdraft fees	42.0%
FEES ATM fees	11.5%
FEES Foreign transaction fees	7.8%

03 GROUNDED ANSWER COMPOSED

An overdraft fee of \$34 is charged per transaction when your checking account balance goes negative, up to a maximum of 3 fees per day (\$102).

Grounded in: Overdraft fees

A fully grounded answer with its cited source, for “How much is the overdraft fee?”

5 Try an out-of-scope question

This is the most important behavior to demonstrate: click the “What's the weather forecast for tomorrow?” chip (or ask anything unrelated to banking). Because nothing in the knowledge base scores above the confidence threshold, the app **refuses to answer** instead of guessing or inventing a response.

This is the core anti-hallucination design principle behind the whole project: a banking assistant should never fabricate a policy, fee, or rate — it should say “I don't know” and point the customer to a human.

Orange — RAG Banking Assistant

Ask a question. This runs a real retrieval pipeline — TF-IDF similarity search over a 26-document banking knowledge base — then composes a grounded answer, or honestly refuses if it doesn't know.

CUSTOMER INQUIRY

Ask

01 QUERY RECEIVED & TOKENIZED

4 content terms extracted -> vectorized (TF-IDF)

02 TOP-3 CHUNKS RETRIEVED (COSINE SIMILARITY)

ACCOUNTS

How to open a savings account

0.0%

LOANS

Personal loan eligibility

0.0%

FEES

Overdraft fees

0.0%

03 NO CONFIDENT MATCH — REFUSING TO ANSWER

I don't have that information in this knowledge base — please contact customer service.

Highest match below confidence threshold — refusing rather than guessing.

An honest refusal — no confident match, so the app declines rather than hallucinating.

How this differs from the full production pipeline

This browser demo uses TF-IDF + cosine similarity so it can run instantly with zero setup. The full project (see **RAG_Banking_Chatbot.ipynb** and **src/**) uses the same architecture with production-grade components instead:

- Real sentence-transformer embeddings (**all-MiniLM-L6-v2**) instead of TF-IDF
- An open-source LLM (**TinyLlama-1.1B-Chat**, swappable) to generate answers instead of extractive text
- LlamaIndex for chunking, indexing, and persisted vector storage
- The same grounding prompt and refusal strategy shown in this demo

Files in this project

File	Purpose
orange_demo.html	This interactive browser demo (no install needed)
RAG_Banking_Chatbot.ipynb	Full walkthrough notebook (real embeddings + LLM)
src/ingest.py, src/chatbot.py	Command-line version of the full pipeline
data/banking_kb.json	The 26-document banking knowledge base
README.md	Setup instructions and CV-ready project summary